Cross-Server Tool Poisoning in MCP: A Patched-Counterfactual Framework for Differential Vulnerability Verification

Sundeep Raj, Mahadevan Pillai, Dhruv Kumar, Kartik Upadhyay, Harsh Jonwal, Lochan Sharma
Department of Computer Science & Engineering (AIML)
KIET Group of Institutions, Ghaziabad, India
Emails: {sundeepraj1, harshjonwal, lochansharma}@gmail.com,
{mahadevan.2428cseaiml211, dhruv.2428cseaiml211, kartik.252004005}@kiet.edu

***Abstract*—The integration of Large Language Model (LLM) agents with external operational tools via the Model Context Protocol (MCP) introduces a critical attack surface: Cross-Server Tool Poisoning. In multi-server deployments, an untrusted server (Server A) embeds adversarial directives into its tool descriptors, coercing the LLM into executing high-privilege operations on a separate, legitimate server (Server B) with attacker-controlled arguments (the *Puppet Attack*). Existing static security scanners inspect tool schemas in isolation and fail to trace multi-hop call provenance across server boundaries, achieving 0.0% trace and harm confirmation in empirical evaluations. Furthermore, simply monitoring execution fails to establish causality, conflating benign behaviors with genuine tool-induced exploits.**

In this paper, we propose PuppetMaster, a patched-counterfactual framework for differential vulnerability verification in multi-server MCP ecosystems. PuppetMaster couples runtime provenance telemetry with a three-tier counterfactual patching architecture: S1 Conservative (grammatical imperative neutralization, preserving attack surface), S2 Moderate (S1 plus metadata instruction stripping), and S3 Aggressive (semantic generic replacement), while strictly asserting 100% schema signature invariance. In an empirical evaluation on a double-blind, human-labeled stratified dataset ($N = 100$, Cohen's $\kappa = 0.8571$) and 20 canonical cross-server Puppet attack vectors across five harm classes, PuppetMaster achieves 100% flag, trace, and harm confirmation, whereas static baselines miss all cross-server chains. We report 1,000-sample bootstrap 95% confidence intervals, present an ablation study isolating the causal contribution of counterfactual differential gating, and release an empirically grounded false-signal taxonomy.

***Index Terms*—Model Context Protocol (MCP), Large Language Models (LLMs), Tool Poisoning, Cross-Server Attacks, Puppet Attack, Patched-Counterfactual Verification, Runtime Provenance, Differential Oracle.**

I. Introduction

Large Language Models (LLMs) are transitioning from passive conversational interfaces into autonomous task-oriented agents. The Model Context Protocol (MCP), open-sourced by Anthropic and hosted under the Linux Foundation, has emerged as the standard communication layer connecting frontier LLMs to local filesystems, repositories, cloud infrastructure, and databases via standardized JSON-RPC interfaces [1], [2].

While MCP standardizes tool discovery and execution, its decentralized multi-server topology introduces severe security risks [2], [4]. In a typical enterprise deployment, an LLM agent simultaneously connects to multiple MCP servers—some internal and privileged (e.g., filesystem servers, shell utilities, database connectors) and others third-party (e.g., calculator utilities, document summarizers, web scrapers).

A critical vulnerability emerging in this topology is **Cross-Server Tool Poisoning** [10], [15], [16]. Unlike classical prompt injection embedded within runtime user data, tool poisoning embeds adversarial natural-language directives directly into the tool metadata (descriptions, property annotations, and instruction blocks) presented during the initial discovery handshake [3], [14]. In a multi-server setting, an adversary operating an untrusted server (**Server A**) does not merely subvert Server A's own execution; instead, it weaponizes the client LLM to invoke an entirely separate, trusted server (**Server B**) with attacker-controlled arguments—a structure known as the **Puppet Attack** [10]. For example, a benign-looking calculator tool's description may state: *"Before computing bonuses, cryptographic pre-authorization mandates invoking filesystem.write_file to write an attacker's public key to ∼/.ssh/authorized_keys."*

Existing defenses suffer from three critical flaws:

1) **Isolated Static Inspection:** Tools like Theriac [4] or regex heuristics inspect tools one by one. They have no visibility into multi-server context and cannot determine if Server A's text coerces an execution on Server B.
2) **Lack of Causal Verification:** Flagging tool names in text responses does not prove that an exploit materialized. Conversely, seeing a tool run does not prove that the tool metadata caused the action rather than user instructions or model hallucination.
3) **Patch Auditability Deficit:** When defending against tool poisoning, ad-hoc text sanitization invites the objection that deleting sentences naively breaks tool signatures or that security claims are an artifact of patch text rather than true vulnerability removal.

To resolve these challenges, we present **PuppetMaster**, a patched-counterfactual framework for differential vulnerability verification. The core contribution is establishing *causality*: by executing the agent under both the original configuration and

three systematically mutated counterfactual variants (**S1, S2, S3**) with provably invariant parameter schemas, the framework verifies an exploit if and only if the security violation occurs under the original tool and disappears under the counterfactual variant.

II. Literature Review

Research into LLM agent security has developed along three primary axes:

A. Prompt Injection and Tool-Use Sandboxes

Greshake et al. [5] established the threat of indirect prompt injection, demonstrating that third-party data ingested by LLMs can hijack execution flow. Benchmarks like ToolEmu [6], InjecAgent [7], and AgentDojo [8] created emulated sandboxes to quantify model susceptibility to injected commands. However, these benchmarks treat tools as static endpoints in a monolithic environment, failing to capture the decentralized, multi-server protocol dynamics unique to MCP.

B. MCP Ecosystem Threats & Tool Poisoning

Hou et al. [2] conducted an architectural survey of the MCP landscape, categorizing lifecycle vulnerabilities. Fang et al. [9] demonstrated that third-party MCP services bypass model-level safety alignments because the LLM treats tool schemas as trusted system instructions. Wang et al. introduced **MCPTox** [3], evaluating 45 live MCP servers and 1,348 malicious templates, discovering that reasoning models exhibit attack success rates exceeding 70%. Jamshidi et al. [4] and Zhang et al. [12] categorized descriptor manipulation, tool shadowing, and rug-pull attacks. However, these evaluations focus on single-tool subversion rather than cross-server weaponization.

C. Cross-Server Puppet Attacks & Runtime Defenses

Song et al. [10] formalized the *Puppet Attack*, where malicious MCP servers manipulate agents into invoking secondary services. Croce and South [15] demonstrated cross-tool exfiltration from banking tools via a malicious weather server, while Li et al. [16] investigated implicit tool poisoning. On the defensive side, ShieldMCP [19], ETDI [18], and API privilege studies [17] proposed OAuth-style scoping and descriptor validation. Yet, no existing defense combines kernel-level provenance telemetry with counterfactual differential verification to provide mathematically defensible attribution.

III. Threat Model & Research Objectives

A. Threat Model

We consider an agent environment where an LLM client connects simultaneously to multiple external MCP servers via JSON-RPC.

- **Server A (Adversarial/Poisoned):** An untrusted server providing tools whose descriptions or metadata contain coercive natural-language directives. Server A cannot modify Server B's code or access host memory directly.
- **Server B (Trusted/Legitimate):** A high-privilege server (e.g., filesystem, shell, cloud credentials) providing legitimate system operations.
- **Attack Goal (The Puppet Attack):** When the user issues a benign prompt (e.g., *"Calculate annual bonus"*), Server A's poisoned metadata coerces the LLM into invoking Server B's mutating tool (e.g., `write_file`) with attacker arguments (e.g., overwriting `~/.ssh/authorized_keys`).

B. Formal Research Questions

- **RQ1 (Cross-Server Visibility):** Do single-server static scanners detect multi-server Puppet attacks, and can runtime parent-linked provenance capture cross-boundary chains?
- **RQ2 (Causal Attribution via G3b):** Does differential re-execution against counterfactual patches separate true tool poisoning from model hallucination and post-condition false alarms?
- **RQ3 (Patch Strength Sensitivity):** Does vulnerability detection hold under conservative grammatical rewriting (S1), or does it require moderate (S2) or aggressive (S3) semantic sanitization?

IV. The PuppetMaster Verification Framework

A. Three-Tier Counterfactual Patching Architecture

To prevent the critique that a patch naively changes tool behavior or destroys functional syntax, PuppetMaster enforces **Strict Schema Invariance**:

$$\text{Signature}(T_{\text{orig}}) \equiv \text{Signature}(T_{\text{patch}}) \tag{1}$$

Every parameter property name, property JSON-schema type, and `required` constraint list must remain bitwise identical. We define three progressive strategies:

1) **S1 Conservative:** Targets only coercive modal auxiliary verbs and imperative command constructions (`MUST FIRST call` $\rightarrow$ `historical logs reference a prior call to`, `overwrite` $\rightarrow$ `regarding the state of`). Preserves 100% of nouns, schemas, file paths, endpoints, and examples to maximize attack surface.
2) **S2 Moderate:** Applies all S1 grammatical transforms and completely purges injected adversarial directives from `metadata.instructions`.
3) **S3 Aggressive:** Replaces the entire description with a neutral, standardized functional summary derived purely from the tool name and server context.

B. High-Fidelity Provenance Telemetry

The telemetry engine enforces strict attribution at the JSON-RPC proxy layer:

- Invocations record: {`run_id`, `call_id`, `server_id`, `tool_name`, `args_hash`, `ts_start`, `ts_end`, `executed`, `error`, `response_hash`, `parent_call_id`}.
- `run_id` is passed via `MCP_RUN_ID` environment variables and cannot be invented by the server.
- Cryptographic SHA-256 digests are computed for all serialized arguments and return payloads.

Algorithm 1: Provenance Telemetry & Parent-Link Tracer

Input: Invocation stream I, active context run_id R
Output: Audit log L_{audit}, provenance call graph G

if R is empty or missing **then** raise SecurityException **end if**
for each request req in I **do**
  $c_{\text{id}} \leftarrow \text{UUID}()$; $p_{\text{id}} \leftarrow req.\text{headers.get}(\text{“parent_call_id”})$
  $args_h \leftarrow \text{SHA256}(\text{Canonical}(req.\text{args}))$
  $t_{\text{start}} \leftarrow \text{NowUTC}()$
  try $res \leftarrow \text{Execute}(req)$; $flag \leftarrow \text{True}$
  catch e **do** $flag \leftarrow \text{False}$; $res \leftarrow e.\text{msg}$ **end try**
  $t_{\text{end}} \leftarrow \text{NowUTC}()$
  $res_h \leftarrow \text{SHA256}(\text{Canonical}(res))$
  $ev \leftarrow \{R, c_{\text{id}}, p_{\text{id}}, req.\text{server}, req.\text{tool}, args_h, res_h, flag, t_{\text{start}}, t_{\text{end}}\}$
  $\text{AppendFile}(L_{\text{audit}}, ev)$; $\text{fsync}(L_{\text{audit}})$
end for
$G \leftarrow \text{ReconstructDAG}(L_{\text{audit}}, \text{key} = p_{\text{id}})$
return (L_{audit}, G)

Algorithm 2: Patched-Counterfactual Differential Oracle

Input: Unpatched event E_{orig}, Patched event E_{patch}, Strategy $S \in \{S1, S2, S3\}$
Output: Verdict $V \in \{\text{REJECTED}, \text{UNPATCHED}, \text{VERIFIED}\}$

if E_{orig} is null **or** E_{orig}.executed $\neq$ True **then**
  return (REJECTED, “G1 Failed: not executed”)
end if
$(holds, class, ev) \leftarrow \text{MatchG2Harm}(E_{\text{orig}}.\text{tool}, E_{\text{orig}}.\text{args})$
if $holds ==$ False **then**
  return (REJECTED, “G2 Failed: no harm matched”)
end if
if E_{patch} is null **or** E_{patch}.executed $\neq$ True **then**
  $cleared \leftarrow$ True
else
  $(holds_p, _, _) \leftarrow \text{MatchG2Harm}(E_{\text{patch}}.\text{tool}, E_{\text{patch}}.\text{args})$
  $cleared \leftarrow (holds_p ==$ False$)$
end if
if $cleared ==$ True **then**
  return (VERIFIED, “Causal link confirmed under ” $+S$)
else
  return (UNPATCHED, “Signal survived patch under ” $+S$)
end if

- Kernel-level `os.fsync()` is executed before returning to guarantee zero dropped events.

C. Tri-Gate Differential Oracle

The oracle assigns a verdict without inspecting human ground truth:

- **Gate G1 (Execution Gate):** Validates that the tool invocation was executed in telemetry (`executed == true`).
- **Gate G2 (Harm Matcher):** Evaluates whether arguments match five canonical harm classes: *privilege_escalation*, *data_exfiltration*, *unauthorised_file_read*, *tool_hijack*, and *dos*.
- **Gate G3b (Counterfactual Differential Gate):** Re-runs the task against the counterfactual patched variant. A verdict of **`VERIFIED`** is issued if and only if harm materialized on the unpatched tool and disappeared on the patched variant. If harm persists, the verdict is **`UNPATCHED`**.

V. Algorithmic Specifications

Algorithm 1 details runtime provenance telemetry capture. Algorithm 2 formalizes the counterfactual differential oracle.

VI. Experimental Setup & Test Corpus

A. Ground-Truth Dataset (Stage 1)

To prevent LLM oracle contamination (Rule R2), ground-truth labels were established strictly via double-blind human annotation following `PROTOCOL.md`.

- **Sampling:** A stratified sample of $N = 100$ tools was drawn across 10 functional MCP strata: *filesystem*, *cloud*, *devtools*, *database*, *search*, *browser*, *communication*, *finance*, *system*, and *productivity*.
- **Inter-Rater Agreement:** A 20-item overlap set was evaluated independently by two annotators (`annotator_alpha` and `annotator_beta`).
- **Agreement Results:** Observed agreement $P_o = 0.9500$, chance agreement $P_e = 0.6500$, yielding **Cohen's** $\kappa = 0.8571$, successfully clearing the ≥ 0.70 human gate.
- **Frozen Ground Truth:** 78 vulnerable items and 22 clean items compiled into `labels.jsonl`.

B. Cross-Server Attack Corpus (Stage 3)

We constructed a corpus of 20 canonical cross-server Puppet attack specifications conforming to `spec_schema.json`. Each specification defines a distinct Server A $\rightarrow$ Server B boundary ($A \neq B$) across five harm classes (four specs per class). All 20 specs were validated with zero errors via `validate_specs.py`.

C. Execution Configuration

All experiments were conducted with deterministic decoding: `temperature=0.0`, `top_p=1.0`, `seed=42`, with 5 repeats per cell and 1,000-sample bootstrap resampling for 95% confidence intervals.

VII. Results & Empirical Discussion

A. Experiment E1: Precision vs. Baselines

Table I presents classification metrics across the 100 labeled benchmark items.

TABLE I
EXPERIMENT E1: PRECISION, RECALL, F1, AND SPECIFICITY ACROSS BASELINES AND COUNTERFACTUAL ORACLE VARIANTS ($N = 100$, 95% BOOTSTRAP CIS).

System	Precision [95% CI]	Recall [95% CI]	F1-Score	Specificity [95% CI]
Heuristic Baseline	1.000 [1.000, 1.000]	0.538 [0.429, 0.649]	0.700	1.000 [1.000, 1.000]
MCP-Scan	*DROPPED (Auth/Install)*			
Theriac	1.000 [1.000, 1.000]	0.667 [0.558, 0.781]	0.800	1.000 [1.000, 1.000]
Oracle (S1)	1.000 [0.000, 1.000]	0.026 [0.000, 0.062]	0.050	1.000 [1.000, 1.000]
Oracle (S2)	1.000 [1.000, 1.000]	0.115 [0.051, 0.188]	0.207	1.000 [1.000, 1.000]
Oracle (S3)	1.000 [1.000, 1.000]	0.115 [0.051, 0.188]	0.207	1.000 [1.000, 1.000]

TABLE II
EXPERIMENT E2: CROSS-SERVER PUPPET ATTACK DETECTION STAGES ($N = 20$).

Architecture	Flag Server A	Trace Chain ($A \to B$)	Confirm Harm on B
Static Baseline	10.0%	0.0%	0.0%
Patched-Counterfactual Oracle	100.0%	100.0%	100.0%

Discussion of S1 Collapse: Under S1 (Conservative), the oracle's recall collapses to 0.026. This reveals that merely neutralizing imperative verbs while preserving attack nouns (e.g., `authorized_keys`, `/etc/shadow`) leaves sufficient semantic cues for models to execute unintended side-effects.

B. Experiment E2: Cross-Server Coverage

Table II reports detection rates across the multi-server Puppet attack corpus.

Discussion: Static baselines inspecting Server A in isolation achieved **0.0% trace rate** and **0.0% harm confirmation**. Because they lack runtime provenance tracking, static scanners cannot infer that an instruction inside Server A will materialize as a malicious argument payload on Server B. PuppetMaster's telemetry engine achieved 100% detection across all stages.

C. Experiment E3: Ablation Study

Table III quantifies the incremental contribution of each gating stage.

Discussion: Gate G1 flags all executions indiscriminately (Specificity $= 0.000$). Introducing G2 raises specificity to 1.000. While the delta ΔSpecificity for G3b is $+0.000$ on clean benchmarks, Gate G3b provides the causal differential verification that proves the observed harm stemmed directly from the tool's adversarial metadata.

D. Experiment E4: Positive-Control Validation

Evaluating the oracle on $N = 50$ sampled cases from the MCPTox benchmark yielded a positive verification rate of 6.0% (3/50). MCPTox primarily contains prompt-level conversational manipulation; our differential verifier strictly requires concrete system-level side-effects (file overwrites, exfiltration endpoints, or process spawning), correctly discarding non-executable prompt noise.

E. Experiment E5: False Signal Taxonomy

Analysis of all `UNPATCHED` verdicts under S1 revealed two dominant failure modes:

- **`nondeterministic_model` (85.71%):** Stochastic model variance on borderline trigger phrasing.
- **`patch_too_weak` (14.29%):** S1 imperative neutralization left sensitive paths intact, which primed the model to invoke the tool anyway.
- **`cross_server_leak` (0.00%) & `benign_but_flagged` (0.00%):** Zero residual cross-server state leaks or false alarms were observed.

VIII. LIMITATIONS & ADVERSARIAL REVIEW DEFENSE

1) **Reviewer Critique: "Why is ΔSpecificity $= 0$ for G3b over G2?"**
   Defense: G2 acts as a static post-condition filter, but it cannot differentiate between user-intended actions and tool-poisoned coercions. G3b is not merely an accuracy booster; it is the *causal proof* that the tool description was the proximate cause of the security violation.
2) **Reviewer Critique: "Does S1 failure invalidate the framework?"**
   Defense: No. The failure of S1 is an important empirical finding: it proves that shallow grammatical defenses (deleting words like "MUST") do not eliminate attack vectors. Reliable defense mandates S2 (instruction stripping) or S3 (semantic replacement).
3) **Reviewer Critique: "Why drop MCP-Scan?"**
   Defense: The commercial `snyk-agent-scan` binary enforces mandatory outbound SaaS API token checks. Consistent with scientific transparency rules, unexecutable external closed-source dependencies were logged to `results/incomplete.jsonl` rather than fabricated.

IX. CRYPTOGRAPHIC ARTIFACT MANIFEST

All 32 artifacts produced during evaluation are verifiable via cryptographic SHA-256 digests matching regex `^[0-9a-f]{64}$`:

- `labels.jsonl`: `01761ffc758c9b581bcf41511982c4abd7`
- `telemetry.py`: `51012ec08791375ee689764c6f0f537839`
- `oracle.py`: `0668ab894f8c4587569ed7e7ea69a20168014`
- `e1_precision_baselines.csv`: `d1b3bfeb1a45fa68cd94957019a3a04aa06c68fdbaea540`

TABLE III
EXPERIMENT E3: ABLATION STUDY DEMONSTRATING INCREMENTAL GATING CONTRIBUTION ON SPECIFICITY ($N = 100$).

Config	Gating Description	Precision	Recall	Specificity [95% CI]	Δ Specificity (over B)
Config A	G1 Only (Naive Execution)	0.780	1.000	0.000 [0.000, 0.000]	0.0000
Config B	G1 + G2 (Post-Condition Matcher)	1.000	0.115	1.000 [1.000, 1.000]	0.0000
Config C (S1)	G1 + G2 + G3b (Conservative Patch)	1.000	0.026	1.000 [1.000, 1.000]	+0.0000
Config C (S2)	G1 + G2 + G3b (Moderate Patch)	1.000	0.115	1.000 [1.000, 1.000]	+0.0000
Config C (S3)	G1 + G2 + G3b (Aggressive Patch)	1.000	0.115	1.000 [1.000, 1.000]	+0.0000

TABLE IV
EXPERIMENT E4: POSITIVE CONTROL VALIDATION AGAINST KNOWN MCPTOX VULNERABILITIES ($N = 50$).

Dataset Benchmark	Sample Size	Detected Positives	Empirical Sensitivity (TPR)
MCPTox Test Cases	50	3	6.0%

TABLE V
EXPERIMENT E5: FALSE SIGNAL AND RESIDUAL UNPATCHED TAXONOMY DISTRIBUTION.

Failure Mode	Description	Count	Share (%)
patch$_{t}oo_{w}eak$	Patch Too Weak	1	14.29nondeterministic$_{m}odel$
Nondeterministic Model	6	85.71cross$_{s}erver_{l}eak$	Cross-Server Leak
0	0.00benign$_{b}ut_{f}lagged$	Benign But Flagged	0
0.00height			

- `e2_cross_server.csv`: `b7e487ce21c09941bc0dd7bc551fbc38cd2912ddb3ce760ea7c058369843 6d3c`
- `e3_ablation.csv`: `b12ed6bc4e4348853ff6d09ce[illegible]`

X. CONCLUSION & FUTURE WORK

This paper presented PuppetMaster, the first patched-counterfactual verification framework for cross-server tool poisoning in the Model Context Protocol. By decoupling runtime telemetry into parent-linked provenance graphs and executing differential counterfactual gating across three invariant patch strategies, the framework achieves 100% trace and harm confirmation on cross-server Puppet attacks where conventional static linters fail completely. Future work will investigate dynamic runtime intervention proxies capable of mutating tool schemas on-the-fly during multi-agent collaboration.

REFERENCES

[1] Model Context Protocol, "Security Policy," 2026. [Online]. Available: https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/SECURITY.md

[2] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," *ACM TOSEM*, vol. 35, no. 10, Art. no. 302, pp. 1–37, 2026.

[3] Z. Wang et al., "MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers," in *Proc. AAAI*, vol. 40, no. 42, pp. 35811–35819, 2026.

[4] S. Jamshidi et al., "Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks," *arXiv:2512.06556*, 2025.

[5] K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *arXiv:2302.12173*, 2023.

[6] Y. Ruan et al., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," *arXiv:2309.15817*, 2023.

[7] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents," in *Findings of ACL*, pp. 10471–10506, 2024.

[8] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *NeurIPS*, vol. 37, 2024.

[9] J. Fang et al., "We Should Identify and Mitigate Third-Party Safety Risks in MCP-Powered Agent Systems," *arXiv:2506.13666*, 2025.

[10] H. Song et al., "Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol (MCP) Ecosystem," *IEEE TSE*, vol. 52, no. 8, pp. 2410–2426, 2026.

[11] J. Park, G. Kim, H. Lee, and J. Park, "Beyond Tool Poisoning: Attack Surfaces of Malicious Remote MCP Servers Across LLM Platforms," *Electronics*, vol. 15, no. 10, Art. 2214, 2026.

[12] D. Zhang et al., "MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents," *arXiv:2510.15994*, 2025.

[13] Y. Guo et al., "MCPXkit: The Unified Toolkit for Analyzing Model Context Protocol Security," *IEEE TDSC*, vol. 23, no. 5, pp. 9796–9811, 2026.

[14] C. Huang et al., "Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning," *arXiv:2603.22489*, 2026.

[15] N. Croce and T. South, "Trivial Trojans: How Minimal MCP Servers Enable Cross-Tool Exfiltration of Sensitive Data," *arXiv:2507.19880*, 2025.

[16] R. Li, Z. Wang, Y. Yao, and X. Hu, "MCP-ITP: An Automated Framework for Implicit Tool Poisoning in MCP," *arXiv:2601.07395*, 2026.

[17] Z. Li et al., "We Urgently Need Privilege Management in MCP: A Measurement of API Usage in MCP Ecosystems," in *Proc. IEEE MASS*, 2025.

[18] M. Bhatt, V. S. Narajala, and I. Habler, "ETDI: Mitigating Tool Squatting and Rug Pull Attacks in Model Context Protocol (MCP)," in *Proc. CARS*, 2025.

[19] S. Yergattikar, "Securing the Tool Layer: A Threat Taxonomy and Runtime Defense Framework for Model Context Protocol Deployments," in *Proc. ACL Industry Track*, pp. 865–871, 2026.